Lotka–Volterra objective

The Lotka–Volterra model describes how two populations affect each other. Without predators, the prey population grows. Without prey, the predator population declines. When both are present, predation reduces the prey population and supports predator growth. Four rates govern these changes. Project prompt

We know the starting populations and have later measurements of both species. The question is which rates could have produced those observations. For any candidate set of rates, we can simulate the populations from the starting state and compare the predictions with the measurements at the same times. Adding up the squared differences gives us one score for comparing candidates.

1. Set up the problem and the loss

Let $x(t)$ be the prey population, $y(t)$ the predator population, and $\boldsymbol{\theta} = (\theta_1, \theta_2, \theta_3, \theta_4)$ the unknown rates. The prompt gives the system and the initial populations.

$$\begin{aligned} \dot{x} &= x(\theta_1 - \theta_2 y), \qquad x(0) = 2.11, \\ \dot{y} &= y(-\theta_3 + \theta_4 x), \quad y(0) = 1.35, \qquad 0 \le t \le 3 \times 10^6. \end{aligned}$$

The search space is the four-dimensional box

$$\Theta = [0.10, 50.0] \times [0.01, 2.00] \times [0.10, 5.00] \times [0.01, 2.00].$$

The CSV contains $N = 27$ rows $(t_n, x_n^{\text{obs}}, y_n^{\text{obs}})$. The first time is 700,000 and the last is 2,960,000. The gaps are irregular and range from 15,000 to 1,100,000. Measurement data

For a given $\boldsymbol{\theta}$, write $x_{\boldsymbol{\theta}}(t)$ and $y_{\boldsymbol{\theta}}(t)$ for the solution of the system. Use the unweighted sum of squares as a starting choice. Each of the 54 residuals gets the same weight.

$$\begin{aligned} J(\boldsymbol{\theta}) &= \sum_{n=1}^{N} \left[\left(x_{\boldsymbol{\theta}}(t_n) - x_n^{\text{obs}}\right)^2 + \left(y_{\boldsymbol{\theta}}(t_n) - y_n^{\text{obs}}\right)^2 \right], \\ \widehat{\boldsymbol{\theta}} &\in \arg\min_{\boldsymbol{\theta} \in \Theta} J(\boldsymbol{\theta}). \end{aligned}$$

The useful least-squares identity is $J = \|\mathbf{r}\|_2^2 = \mathbf{r}^{\mathsf{T}}\mathbf{r}$, where $\mathbf{r}$ stacks the prey and predator residuals. The normal equations on the linear regression slide require a fixed design matrix. Here the predicted populations depend nonlinearly on the rates through the ODE solution, so we evaluate the loss by solving the system for each candidate.

- Lecture 13, slides 4–5 gives the residual norm and the fixed-design least-squares problem.
- Lecture 20, slides 2 and 4 connects a bounded search space to a scalar objective evaluated at each query.

2. Integrate once per objective call

Put the two populations in one state vector, $\mathbf{z} = (x, y)^\mathsf{T}$, and call the right-hand side $\mathbf{F}(\mathbf{z}; \boldsymbol{\theta})$. Both populations must be updated together at each stage. The system is autonomous, which means $\mathbf{F}$ has no explicit time argument. For a step of length h, classical RK4 gives

$$\begin{aligned}
\mathbf{k}_1 &= \mathbf{F}(\mathbf{z}; \boldsymbol{\theta}), \\
\mathbf{k}_2 &= \mathbf{F}(\mathbf{z} + \tfrac{h}{2}\mathbf{k}_1; \boldsymbol{\theta}), \\
\mathbf{k}_3 &= \mathbf{F}(\mathbf{z} + \tfrac{h}{2}\mathbf{k}_2; \boldsymbol{\theta}), \\
\mathbf{k}_4 &= \mathbf{F}(\mathbf{z} + h\mathbf{k}_3; \boldsymbol{\theta}), \\
\mathbf{z}_{\text{next}} &= \mathbf{z} + \frac{h}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right).
\end{aligned}$$

1. Start at $t = 0$ with $\mathbf{z} = (2.11, 1.35)^\mathsf{T}$.
2. Step forward to the first measurement, then carry the current state forward from one measurement time to the next.
3. Treat t_n as an output time. With fixed-step RK4, shorten the last internal step in each interval to land on t_n.
4. Store the two predicted values at each t_n and compute J from those 27 pairs. The objective can stop at the last measurement. Extend the same solver to 3×10^6 for a trajectory over the full interval.

A single forward solve gives all the predictions. Keeping only the current state and the requested outputs also avoids storing millions of internal steps.

3. Check the solver before timing it

The positive equilibrium follows by setting both derivatives to zero.

$$(x^*, y^*) = \left(\frac{\theta_3}{\theta_4}, \frac{\theta_1}{\theta_2}\right).$$

For example, $\boldsymbol{\theta} = (1.35, 1, 2.11, 1)$ lies inside the box and makes the given initial state an equilibrium. The solver should return the same state at every time for this test.

The equations also give a conserved quantity for positive x and y.

$$\begin{aligned}
H(x, y) &= \theta_4 x - \theta_3 \log x + \theta_2 y - \theta_1 \log y, \\
\frac{dH}{dt} &= (\theta_4 x - \theta_3)(\theta_1 - \theta_2 y) + (\theta_2 y - \theta_1)(\theta_4 x - \theta_3) = 0.
\end{aligned}$$

For a nonconstant trajectory, compare a short RK4 run against MATLAB's `ode45` or SciPy's `solve_ivp`. Check that the populations stay positive and see how much H drifts when you halve h. Then compare the predictions and J at the actual measurement times with tighter integration settings. A small error in the oscillation period can produce a large phase error by the last measurement, even when the orbit itself looks right.

Why the long horizon matters

Even $h = 0.01$ would take about 296 million steps to reach the last measurement. That number illustrates the cost. Choose the step size or tolerance by checking the accuracy of the predictions and loss. If fixed-step RK4 is too slow at adequate accuracy, an adaptive Runge–Kutta method is a natural next choice. Reusing a periodic orbit is another possible shortcut if the instructor accepts it. A period error accumulates over the number of elapsed cycles, so check the phase at the actual measurement times before relying on that shortcut.

One exact identity explains why phase also matters for estimating the rates. With the same initial state and $c > 0$, scaling all four rates changes the speed along the trajectory.

$$\mathbf{z}(t; c\boldsymbol{\theta}) = \mathbf{z}(ct; \boldsymbol{\theta}).$$

For a non-equilibrium periodic trajectory with period P, the CSV times are all integer multiples of $\Delta = 5{,}000$. If $c = 1 + mP/\Delta$ for an integer m, then $(c-1)t_n$ is an integer number of periods for every row. Whenever $c > 0$ and both parameter vectors lie in the box, the model gives the same predictions at those times and the same least-squares loss. This is a consequence of the model and sampling times and matters when interpreting later fits. A low loss can have several rate vectors behind it.

4. Time one objective evaluation

Load the CSV first. Pick a non-equilibrium parameter vector inside the box and record it. Time one call that integrates the system at that vector and computes J. Leave file loading and plotting outside the timer so the measurement reflects what one optimization query costs.

Keep the parameter vector, solver settings, objective value, and wall time together. Include the number of internal steps or right-hand-side evaluations if available. This gives the report enough detail to say what was timed and to compare later timings under the same conditions.

Online implementation references

- Classical RK4 stages and Python example, College of Charleston
- MATLAB `ode45` for a Runge–Kutta reference solve and `odeset` for tolerances and events.
- MATLAB `tic` and `toc` for timing a single call.
- SciPy `solve_ivp` for RK45, requested output times, and tolerances.
- Python `time.perf_counter` for elapsed time.